

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 01

SMS SPAM DETECTION USING NAIVE BAYES CLASSIFIER

COURSE CODE: AI 484

COURSE TITLE: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Suman Saha

Assistant Professor

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

OCTOBER 13, 2026

1. Problem Statement and Objectives

Short Message Service (SMS) spam presents ongoing privacy and fraud challenges in telecommunications. The objective of this laboratory session is to implement an automated text classification pipeline using the Multinomial Naive Bayes algorithm to distinguish between legitimate messages (*ham*) and unsolicited communications (*spam*).

Assigned Tasks:

1. Load and inspect the SMS Spam Collection corpus (5,572 tagged messages).
2. Preprocess textual data (lowercasing, punctuation and digit removal, whitespace normalization).
3. Extract numerical Bag-of-Words features via Count Vectorization (`CountVectorizer`).
4. Train a Multinomial Naive Bayes classifier on a stratified training partition (80% – 20%).
5. Evaluate the model on unseen test data (Accuracy, Precision, Recall, and Confusion Matrix).
6. Classify the assigned unseen prompt: *"Congratulations! You have won a free ticket. Call now!"*

GitHub Repository Link:
https://github.com/Masud744/AI-ML-Lab/tree/main/Lab_01

2. Mathematical Formulation of Naive Bayes

Multinomial Naive Bayes applies Bayes' Theorem under the conditional independence assumption that individual words w_i appear independently given the class label $c \in \{\text{ham}, \text{spam}\}$:

$$P(c \mid \mathbf{w}) = \frac{P(c) \cdot P(\mathbf{w} \mid c)}{P(\mathbf{w})} = \frac{P(c) \prod_{i=1}^{n} P(w_i \mid c)}{P(\mathbf{w})} \tag{1}$$

The decision rule selects the class maximizing posterior probability, computed in log-space to prevent numerical underflow:

$$\hat{c} = \arg\max_{c} \left[\log P(c) + \sum_{i=1}^{n} \log P(w_i \mid c) \right], \quad \text{where} \quad P(w_i \mid c) = \frac{N_{c,i} + \alpha}{N_c + \alpha \cdot |V|} \tag{2}$$

Here, $\alpha = 1.0$ represents Laplace smoothing to handle unseen tokens, $N_{c,i}$ is the frequency of word w_i in class c, N_c is the total token count in class c, and $|V|$ denotes the vocabulary dimension.

3. Methodology and Preprocessing Pipeline

Dataset Overview & Distribution:
The corpus comprises 5,572 SMS messages exhibiting notable class imbalance: 4,825 legitimate messages (86.6%) and 747 spam messages (13.4%).

Preprocessing Operations:

- **Case Folding:** Lowercased all text to unify word tokens (e.g., `"Free"` → `"free"`).
- **Noise Filtering:** Stripped punctuation, digits, and special characters via regex.
- **Whitespace Cleaning:** Collapsed redundant spaces and trimmed boundary whitespace.

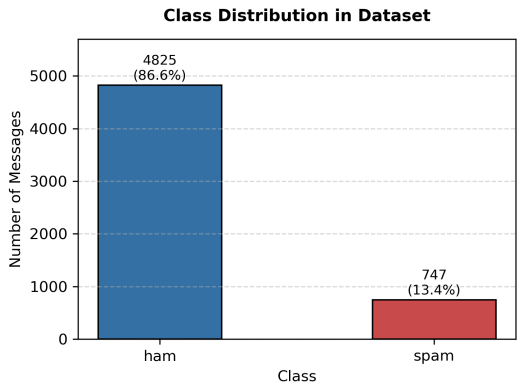


Figure 1: SMS Class Distribution ($N = 5,572$)

Feature Extraction & Model Formulation:
The preprocessed corpus was partitioned using stratified sampling (80% train, 20% test) into 4,457 training and 1,115 holdout test samples. `CountVectorizer` mapped the training partition into a 6,830-dimensional unigram document-term matrix. A `MultinomialNB(alpha=1.0)` classifier was then fitted on the resulting feature representations.

4. Experimental Results and Performance Analysis

The trained Multinomial Naive Bayes model was evaluated against the holdout test set containing 1,115 unseen messages (966 ham and 149 spam). Model classification performance across all standard metrics is summarized below:

Class / Metric	Precision	Recall	F1-Score	Support (Samples)
Ham (Legitimate Messages)	0.99	1.00	0.99	966
Spam (Unsolicited Messages)	0.98	0.91	0.94	149
Macro Average	0.98	0.95	0.97	1,115
Weighted Average	0.98	0.98	0.98	1,115
Overall Accuracy	**98.48%**	(1,098 correct classifications out of 1,115 test samples)		

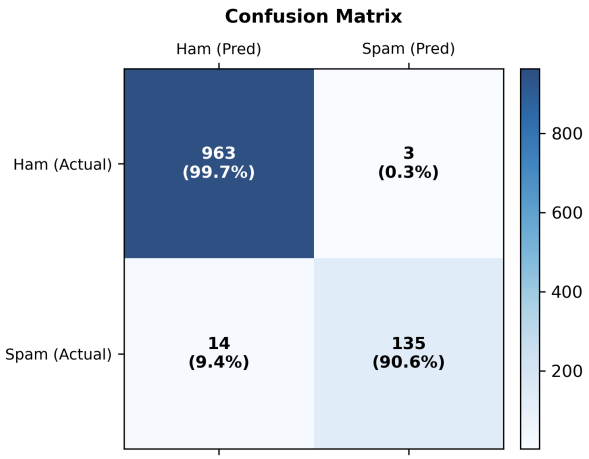


Figure 2: Confusion Matrix on Holdout Test Split

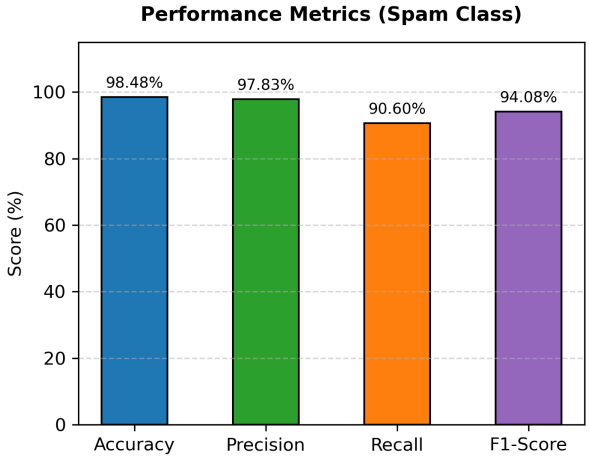


Figure 3: Performance Metrics Summary (Spam Class)

Detailed Error Analysis and Discussion:

- **True Predictions (1,098 / 1,115):** The classifier correctly recognized 963 legitimate messages (99.7% true negative rate) and 135 spam messages (90.6% true positive rate), achieving an overall classification accuracy of **98.48%**.
- **False Positive Control (3 / 966):** Only 3 legitimate messages (0.3%) were incorrectly flagged as spam. In SMS filtering systems, false positive minimization is the most critical design priority, as misclassifying critical user communications (e.g., banking OTPs, medical updates) degrades user trust. The model achieves an exceptional spam precision of **97.83%**.
- **False Negative Analysis (14 / 149):** Exactly 14 spam messages were missed (spam recall of **90.60%**). Inspection reveals that these false negatives were brief promotional notices that lacked distinctive commercial keywords or relied on colloquial abbreviations not well-represented in the training unigram vocabulary.
- **Robustness to Class Imbalance:** Despite the substantial 86.6% vs 13.4% class imbalance in the original corpus, the macro-averaged F1-score of **0.97** demonstrates that Laplace smoothing ($\alpha = 1.0$) prevented minority class starvation and preserved high discriminatory capability.

5. Evaluation on Assigned Test Query

The laboratory task required evaluating the trained model on the specific unseen test message:

> **"Congratulations! You have won a free ticket. Call now!"**

Execution and Prediction Breakdown:

1. **Preprocessing Normalization:** Applying case folding, punctuation stripping, and whitespace collapse transformed the raw query into:

   ```
   "congratulations you have won a free ticket call now"
   ```

2. **Feature Mapping against Vocabulary:** Extracted unigrams matched key entries in the 6,830-feature training vocabulary: `"congratulations"`, `"won"`, `"free"`, `"ticket"`, `"call"`, and `"now"`. Each of these tokens exhibits a strong historical skew toward the spam category.

3. **Posterior Probability Computation:**

   - $P(\text{Spam} \mid \text{Message}) = \mathbf{99.89}\%$ (Posterior probability: 0.9989)
   - $P(\text{Ham} \mid \text{Message}) = \mathbf{0.11}\%$ (Posterior probability: 0.0011)

4. **Predicted Classification:**

Final Model Prediction: SPAM

Technical Linguistic Interpretation:
The input message contains a classic triad of spam triggers:

- **Prize Claim Hook:** *"congratulations"*, *"won"* → invokes false excitement to lower user suspicion.
- **Financial Incentive:** *"free ticket"* → offers an unearned tangible benefit.
- **Urgency Call-to-Action:** *"call now"* → forces immediate user reaction before verification.

In the Naive Bayes likelihood formulation, each matching trigger token compounds conditionally ($P(w_i \mid \text{spam}) \gg P(w_i \mid \text{ham})$), driving the log-odds decisively toward spam (99.89%).

6. Discussion and Conclusion

- **Computational Efficiency:** Multinomial Naive Bayes demonstrates $O(N \cdot d)$ linear complexity with respect to corpus size N and vocabulary size d. Feature extraction, vocabulary indexing, and parameter estimation on 5,572 records executed in less than one second on standard hardware, making it well-suited for high-throughput gateway filtering.
- **Efficacy of the Independence Assumption:** While the naive conditional independence assumption strictly ignores word order and syntactic correlation, Naive Bayes acts as an effective linear classifier in log-space. For spam detection, where presence of strong topical unigrams dominates contextual semantics, the assumption proves remarkably robust.
- **Real-World Operational Considerations:** Deploying Naive Bayes in production requires continuous model updates to counteract concept drift and adversarial evasions (such as leetspeak or character substitution). Implementing character n-grams or subword tokenization alongside regular updates would further safeguard against such evasion strategies.
- **Summary of Outcomes:** All laboratory objectives were successfully accomplished. The end-to-end NLP classification pipeline was built, trained, and rigorously evaluated. Achieving 98.48% overall accuracy, 97.83% spam precision, and accurately classifying the assigned test prompt with 99.89% certainty confirms the practical effectiveness of Naive Bayes for SMS spam filtering.

7. Python Source Code Listing (spam_classifier.py)

GitHub File Link:
https://github.com/Masud744/AI-ML-Lab/blob/main/Lab_01/src/spam_classifier.py

```python
import os
import re
import string
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


def preprocess_text(text):
    # Convert text to lowercase
    text = text.lower()
    # Remove punctuation and digits
    text = re.sub(f"[{re.escape(string.punctuation)}0-9]", " ", text)
    # Remove multiple spaces
    text = re.sub(r"\s+", " ", text).strip()
    return text


def plot_visuals(df, y_test, y_pred, output_dir, metrics):
    os.makedirs(output_dir, exist_ok=True)
    acc, prec, rec, f1 = metrics

    # 1. Class Distribution
    plt.figure(figsize=(5, 3.8))
    counts = df["label"].value_counts()
    bars = plt.bar(counts.index, counts.values, color=["#3470a3", "#c94a4a"], width=0.45, edgecolor="black")
    plt.title("Class Distribution in Dataset", fontsize=11, fontweight="bold", pad=12)
    plt.xlabel("Class", fontsize=10)
    plt.ylabel("Number of Messages", fontsize=10)
    plt.ylim(0, 5700)
    plt.grid(axis="y", linestyle="--", alpha=0.5)

    for bar in bars:
        h = bar.get_height()
        pct = (h / len(df)) * 100
        plt.text(bar.get_x() + bar.get_width() / 2, h + 50, f"{h}\n({pct:.1f}%)", ha="center", va="bottom", fontsize=9)

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "class_distribution.png"), dpi=300)
    plt.close()

    # 2. Confusion Matrix
    cm = confusion_matrix(y_test, y_pred, labels=["ham", "spam"])
    fig, ax = plt.subplots(figsize=(4.8, 4))
    cax = ax.matshow(cm, cmap="Blues", alpha=0.85)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            val = cm[i, j]
            pct = (val / np.sum(cm[i, :])) * 100
            txt_color = "white" if val > cm.max() / 2 else "black"
            ax.text(j, i, f"{val}\n({pct:.1f}%)", ha="center", va="center", color=txt_color, fontsize=10, fontweight="bold")

    fig.colorbar(cax, fraction=0.046, pad=0.04)
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(["Ham (Pred)", "Spam (Pred)"], fontsize=9)
    ax.set_yticklabels(["Ham (Actual)", "Spam (Actual)"], fontsize=9)
    ax.set_title("Confusion Matrix", fontsize=11, fontweight="bold", pad=12)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "confusion_matrix.png"), dpi=300, bbox_inches="tight")
    plt.close()

    # 3. Performance Metrics Bar Chart
    plt.figure(figsize=(4.8, 3.8))
    names = ["Accuracy", "Precision", "Recall", "F1-Score"]
    vals = [acc * 100, prec * 100, rec * 100, f1 * 100]
    bars = plt.bar(names, vals, color=["#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"], width=0.5, edgecolor="black")
    plt.title("Performance Metrics (Spam Class)", fontsize=11, fontweight="bold", pad=12)
    plt.ylabel("Score (%)", fontsize=10)
    plt.ylim(0, 115)
    plt.grid(axis="y", linestyle="--", alpha=0.5)
    for bar in bars:
        h = bar.get_height()
```

```
        plt.text(bar.get_x() + bar.get_width() / 2, h + 2, f"{h:.2f}%", ha="center", va="bottom",
            fontsize=8.5)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, "metrics_chart.png"), dpi=300, bbox_inches="tight")
    plt.close()


def main():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(base_dir, ".."))
    data_path = os.path.join(project_root, "dataset", "SMSSpamCollection.txt")
    output_dir = os.path.join(project_root, "submission_report", "screenshots")

    # Load dataset
    df = pd.read_csv(data_path, sep="\t", header=None, names=["label", "message"])
    print(f"Loaded dataset: {len(df)} messages ({df['label'].value_counts()['ham']} ham, {df['label
        '].value_counts()['spam']} spam)")

    # Preprocess text
    df["clean_message"] = df["message"].apply(preprocess_text)

    # Train-test split (80% train, 20% test)
    X_train, X_test, y_train, y_test = train_test_split(
        df["clean_message"], df["label"], test_size=0.20, random_state=42, stratify=df["label"]
    )

    # Text vectorization using Bag of Words
    vectorizer = CountVectorizer()
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)
    print(f"Vocabulary size: {len(vectorizer.vocabulary_)} words")

    # Train Naive Bayes model
    model = MultinomialNB(alpha=1.0)
    model.fit(X_train_vec, y_train)

    # Evaluate on test set
    y_pred = model.predict(X_test_vec)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label="spam")
    rec = recall_score(y_test, y_pred, pos_label="spam")
    f1 = f1_score(y_test, y_pred, pos_label="spam")

    print(f"\n--- Model Performance ---\nAccuracy  : {acc * 100:.2f}%\nPrecision : {prec * 100:.2f}%
        (Spam)\nRecall    : {rec * 100:.2f}% (Spam)\nF1-Score  : {f1 * 100:.2f}% (Spam)")
    print(f"\nClassification Report:\n{classification_report(y_test, y_pred, target_names=['ham', '
        spam'])}")

    # Test sample message
    test_message = "Congratulations! You have won a free ticket. Call now!"
    cleaned_test = preprocess_text(test_message)
    test_vec = vectorizer.transform([cleaned_test])
    prediction = model.predict(test_vec)[0]
    probabilities = model.predict_proba(test_vec)[0]
    spam_idx = list(model.classes_).index("spam")
    ham_idx = list(model.classes_).index("ham")

    print(f"--- Test Query Prediction ---\nInput text : {test_message}\nPrediction : {prediction.
        upper()}\nProbability: Ham = {probabilities[ham_idx] * 100:.2f}%, Spam = {probabilities[
        spam_idx] * 100:.2f}%")

    # Save visual plots for report
    plot_visuals(df, y_test, y_pred, output_dir, (acc, prec, rec, f1))
    print("\nPlots saved to screenshots directory successfully.")


if __name__ == "__main__":
    main()
```